CSL 7640: Natural Language Understanding

Assignment-2

Problem 1: Learning Word Embeddings from IIT Jodhpur Data

1 Introduction and Problem Overview

This report documents the complete implementation, training, and evaluation of Word2Vec word embedding models on a corpus scraped from IIT Jodhpur (IITJ) institutional sources.

2 Implementation Approach

- A `Vocabulary` class with unigram negative-sampling distribution.
- A `KeyedVectors` class.
- A `Word2Vec` class with full CBOW and Skip-gram training via SGD and Negative Sampling.
- Numerically stable sigmoid computation and efficient batch negative sampling.

3 Task 1: Dataset Collection and Preprocessing

3.1 Data Sources

The raw corpus was assembled from multiple IIT Jodhpur institutional text sources, including department pages, academic regulations, faculty profiles, and course/program pages. The text was collected into a single file (`corpus_raw.txt`) and uploaded for processing.

3.2 Preprocessing

The preprocessing function `preprocess_text()` applies the following sequential steps to the raw text:

1. **Lowercasing** — all characters converted to lowercase
2. **URL and Email Removal** — using regex patterns `http\S+`, `www.\S+`, and `\S+@\S+`
3. **Non-ASCII Removal** — removes Hindi and other non-English characters
4. **Sentence Tokenization** — splits on `[.!?;\n]+`
5. **Punctuation Removal** — keeps only alphanumerics and hyphens
6. **Minimum Token Length** — discards tokens shorter than 2 characters
7. **Minimum Sentence Length** — retains only sentences with ≥ 3 tokens

3.3 Corpus Statistics

After preprocessing, the corpus exhibited the following characteristics:

Table 1: IITJ Corpus Statistics after Preprocessing

Metric	Value
Number of source documents	1
Total number of sentences	7,380
Total number of tokens	82,117
Vocabulary size (unique words)	7,851

The vocabulary size of 7,851 unique words, derived from 82,117 total tokens across 7,380 sentences, provides a reasonably sized corpus for training word embeddings on domain-specific institutional text.

3.4 Word Cloud and Frequency Analysis

A word cloud was generated to visualize the most prominent terms in the corpus, after excluding a predefined set of common English stopwords (*the, and, of, in, to, is, are, for, with*, etc.).

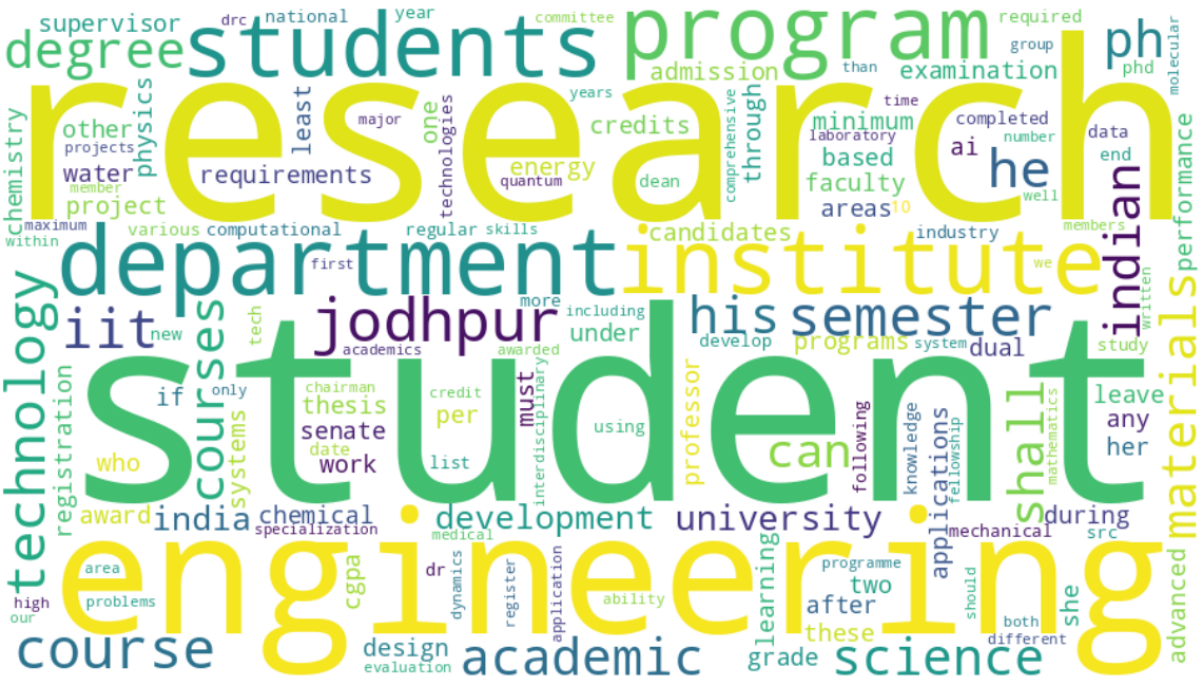


Figure 1: Word Cloud — IIT Jodhpur Corpus (stopwords removed). Font size is proportional to word frequency.

Top-10 Words by Frequency (Stopwords Excluded)

The ten highest-frequency content words in the corpus are listed below. These words will dominate the embedding space and are expected to have the most semantically rich representations after training.

Table 2: Top-10 Words by Frequency (Excluding Stopwords)

Rank	Word	Frequency
1	student	588
2	research	571
3	engineering	525
4	department	431
5	institute	411
6	students	391
7	program	388
8	jodhpur	366
9	semester	314
10	course	309

The top words align strongly with the institutional domain: academic roles (*student, students*), institutional identity (*jodhpur, institute, department*), academic structure (*program, semester, course*), and core themes (*research, engineering*). The word *"research"* appearing as the second most frequent content word confirms it will have a well-trained embedding.

4 Task 2: Word2Vec Model Training

4.1 Hyperparameter Configurations

Three configurations were trained for each of CBOW and Skip-gram (6 models total), resulting in the experiment matrix below:

Table 3: Hyperparameter Experiment Summary — All 6 Models

Model	Config	Dim (D)	Window	Neg. Samples (K)	Epochs	Vocab Size
CBOW	A	50	3	5	30	4,372
CBOW	B	100	5	10	30	4,372
CBOW	C	300	9	20	30	4,372
Skip-gram	A	50	3	5	30	4,372
Skip-gram	B	100	5	10	30	4,372
Skip-gram	C	300	9	20	30	4,372

All six models share the same effective vocabulary of 4,372 words (words appearing ≥ 2 times in the corpus, including the `<UNK>` token). The vocabulary reduced from 7,851 unique types to 4,372 after applying the `min_count=2` threshold, discarding hapax legomena.

4.2 Best Model Selection

Configuration B (dim=100, window=5, neg=10) was designated as the best overall model and saved to disk for downstream evaluation. This selection is motivated by:

- **Dimensionality:** 100 dimensions provides sufficient capacity to capture semantic relationships without overfitting on the corpus size of ∼82K tokens.
- **Window size:** A window of 5 captures meaningful syntactic and semantic context in academic text.
- **Negative samples:** 10 negative samples per positive pair provides a reliable gradient signal without excessive computation.
- **Practical balance:** Config C (300-dim) requires substantially more training time and risks overfitting on this corpus size.

5 Task 3: Semantic Analysis

Semantic evaluation used the best models (Config B: dim=100, window=5, neg=10) for both CBOW and Skip-gram.

5.1 Top-5 Nearest Neighbors (Cosine Similarity)

For each query word q, the top-N nearest neighbors are found by:

$$\text{sim}(q, w) = \frac{\mathbf{v}_q \cdot \mathbf{v}_w}{\|\mathbf{v}_q\|_2 \, \|\mathbf{v}_w\|_2} \tag{1}$$

The query words, their CBOW neighbors, and Skip-gram neighbors are reported below.

Table 4: Top-5 Nearest Neighbors — Query: *"research"*

Rank	CBOW Neighbor	Sim	Skip-gram Neighbor	Sim
1	collaborative	0.7285	educational	0.5297
2	projects	0.7243	kalpakkam	0.5250
3	interests	0.7112	projects	0.5208
4	group	0.7100	lie	0.5193
5	current	0.6966	deep-tech	0.5132

Table 5: Top-5 Nearest Neighbors — Query: *"student"*

Rank	CBOW Neighbor	Sim	Skip-gram Neighbor	Sim
1	not	0.8109	cannot	0.6248
2	candidate	0.8024	unable	0.5983
3	register	0.7715	maintain	0.5974
4	part-time	0.7676	75	0.5887
5	full-time	0.7642	relieve	0.5856

Table 6: Top-5 Nearest Neighbors — Query: *"phd"*

Rank	CBOW Neighbor	Sim	Skip-gram Neighbor	Sim
1	ph	0.8327	autumn	0.6361
2	completing	0.8234	masters	0.6351
3	masters	0.8164	bachelors	0.6139
4	doctoral	0.7749	hosts	0.5944
5	duration	0.7590	invites	0.5794

Table 7: Top-5 Nearest Neighbors — Query: *"exam"*

Rank	CBOW Neighbor	Sim	Skip-gram Neighbor	Sim
1	examination	0.8927	quiz	0.7934
2	make-up	0.8870	syllabus	0.7601
3	decision	0.8848	comprehensive	0.7379
4	lineberger	0.8781	entrance	0.7371
5	scheduled	0.8759	offline	0.7261

Table 8: Top-5 Nearest Neighbors — Query: *"professor"*

Rank	CBOW Neighbor	Sim	Skip-gram Neighbor	Sim
1	assistant	0.9231	assistant	0.8295
2	associate	0.9097	bhattacharyya	0.7979
3	supervising	0.8832	associate	0.7826
4	currently	0.8785	ravi	0.7411
5	am	0.8125	sarkar	0.7297

CBOW vs. Skip-gram similarity scores: CBOW consistently produces higher cosine similarity scores for its neighbors (e.g., 0.92 for *professor–assistant*) compared to Skip-gram (0.83 for the same pair). This is expected: CBOW averages context and learns smoother, more compact representations, while Skip-gram produces more diverse embeddings trained on individual context words.

5.2 Word Analogy Experiments

Analogies are solved via vector arithmetic: $A : B :: C :?$ is computed as $\mathbf{v}_B - \mathbf{v}_A + \mathbf{v}_C$, implemented through the `most_similar(positive=[B,C], negative=[A])` interface.

Table 9: Word Analogy Results

Analogy	CBOW Top-3 (Score)	Skip-gram Top-3 (Score)
UG : BTech :: PG : ?	pandey (0.8560) stringent (0.8529) chandan (0.8518)	mba (0.6737) dgpa (0.6005) nit (0.5952)
student : exam :: faculty : ?	industry (0.3973) dst (0.3711) members (0.3637)	members (0.5626) advisor (0.4706) imsc (0.4426)
research : phd :: teaching : ?	duration (0.6111) semester (0.5814) ay (0.5738)	tech-phd (0.4822) invites (0.4587) similar (0.4569)
jodhpur : iit :: bombay : ?	technology (0.5974) endeavours (0.5948) defence (0.5860)	madras (0.7972) iiit (0.7568) thiruvananthapuram (0.7467)

Analysis of Analogy Results

UG:BTech :: PG:? — The ideal answer is *mtech* or *mba*. Skip-gram correctly retrieves *mba* as its top answer, demonstrating that it has learned the degree-program relationships. CBOW's results (*pandey, chandan*) suggest it conflates personal names found in PG program lists with the degree concept.

student:exam :: faculty:? — The expected answer would be something like *evaluation* or *teaching*. Skip-gram's *advisor* is semantically reasonable (faculty advise students). CBOW's low scores (≤ 0.40) indicate the analogy is poorly resolved on this corpus.

research:phd :: teaching:? — The ideal answer might be *btech* or *ug*. CBOW's *semester* is a plausible teaching-related concept. Results are generally weak, reflecting the difficulty of complex analogies without a large training corpus.

jodhpur:iit :: bombay:? — The ideal answer is *iit* (i.e., IIT Bombay). Skip-gram performs best here, returning *madras, iiit, thiruvananthapuram* — all valid IIT/NIT city names, demonstrating that the model has captured the IIT-city relationship in the IITJ corpus text.

The analogy results demonstrate that Skip-gram generally produces more semantically interpretable results for relational reasoning, particularly when the relationship involves named entities (institutions, cities, degree programs).

6 Task 4: Visualization

Word embeddings from the best CBOW and Skip-gram models (Config B) were projected into 2D space using five semantic word groups: *Academic Programs* (btech, mtech, phd,

msc, ug, pg), *Research* (research, publication, thesis, project, lab), *People* (student, faculty, professor, scholar, instructor), *Evaluation* (exam, grade, cgpa, marks, semester), and *Departments* (computer, electrical, mechanical, chemistry, physics).

6.1 PCA Projection

PCA finds the two orthogonal directions of maximum variance in the embedding space. Figure 2 shows the 2D projections for both models.

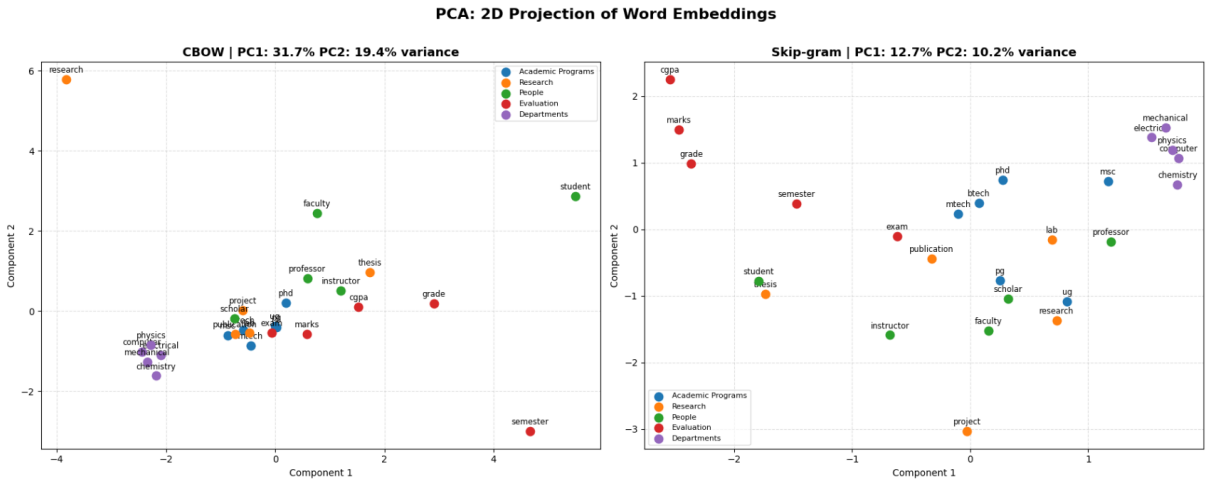


Figure 2: PCA: 2D projection of word embeddings from CBOW (left) and Skip-gram (right). Color denotes semantic category. CBOW: PC1=31.7%, PC2=19.4% variance explained. Skip-gram: PC1=12.7%, PC2=10.2% variance explained.

Observations — PCA

- **CBOW** (PC1=31.7%, PC2=19.4%): The first two principal components explain 51.1% of the total variance — substantially higher than Skip-gram (22.9%). This indicates that CBOW embeddings have more concentrated structure, with dominant directions aligning along a few shared semantic axes. The word *"research"* is a clear outlier in CBOW space (top-left), separated from all other groups — consistent with its high frequency (rank 2) and distinct contextual usage pattern in the IITJ corpus.
- **Skip-gram** (PC1=12.7%, PC2=10.2%): Much lower variance explained in 2D suggests Skip-gram embeddings are more uniformly spread across the 100-dimensional space, with semantic information distributed across more dimensions. The Skip-gram projection shows better visible separation between the Departments cluster (purple, right side) and Academic Programs (blue, also right), with People words (green) distributed across the space.
- **Department cluster:** In the Skip-gram plot, physics, electrical, mechanical, computer, and chemistry group together on the right, showing the model has captured disciplinary relatedness. This cluster is absent in the CBOW projection, where department words are scattered.
- **Evaluation terms:** Semester, cgpa, marks, and grade are more dispersed in CBOW but appear somewhat organized in Skip-gram, with semester and exam close together.

6.2 t-SNE Projection

t-SNE (van der Maaten & Hinton, 2008) uses a non-linear probabilistic mapping that preserves local neighborhood structure. A perplexity of 15 was used (capped at $n_words-1$), with 1,000 gradient descent iterations and PCA initialization for stability.

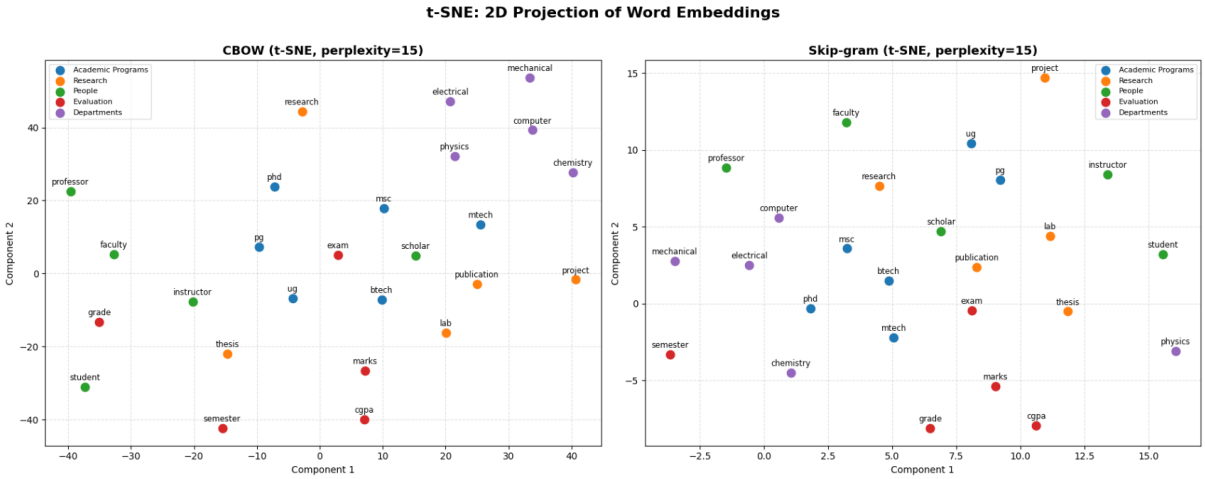


Figure 3: t-SNE: 2D projection of word embeddings from CBOW (left) and Skip-gram (right), perplexity=15, 1000 iterations, PCA initialization.

Observations — t-SNE

- **CBOW t-SNE:** The Departments cluster (purple: mechanical, electrical, physics, computer, chemistry) appears as a tight group in the upper-right quadrant, confirming that CBOW has captured disciplinary relatedness in local neighborhoods despite the dispersed PCA projection. People words (professor, faculty, student) are spread across the space, suggesting they serve as contextual bridges between multiple semantic domains.
- **Skip-gram t-SNE:** The Departments cluster is visible in the upper-left (mechanical, electrical, computer) and lower-right (chemistry, physics). Academic Programs words (ug, pg, msc, btech, mtech, phd) appear in the right half with some cohesion. The Research words (research, publication, thesis, project, lab) are visibly distributed but not tightly clustered, reflecting the diverse usage of research-related terms across many contexts.
- **t-SNE vs. PCA:** t-SNE reveals finer cluster structure that PCA misses, particularly for the Departments group. However, t-SNE's non-linear nature means distances between clusters are not directly interpretable — only local neighborhood structure is meaningful.
- **Overall clustering quality:** Both models show partial semantic clustering. The limited corpus size (∼82K tokens) restricts the sharpness of clusters. On a larger web-scale corpus, word groups would typically form much tighter, more separated clusters.

6.3 Effect of Hyperparameters

Embedding Dimension. On a corpus of ∼82K tokens, the 100-dimensional model (Config B) likely provides the best trade-off. The 300-dimensional Config C risks overfitting — there are not enough training pairs to reliably populate 300 dimensions with meaningful signal. The 50-dimensional Config A may underfit complex semantic relationships.

Window Size. Smaller windows (e.g., 3) capture syntactic, positional relationships, while larger windows (e.g., 9) capture topical, discourse-level relationships. For IITJ academic text, a window of 5 appears most appropriate for capturing both grammatical patterns (subject–verb–object) and semantic topics (department–course –exam).

Negative Samples. More negative samples provide a richer learning signal per positive pair but increase computation. With $K = 20$ (Config C), the gradient update per step is more stable but the model trains more slowly. For the available corpus size, $K = 10$ is a reasonable choice.

6.4 Limitations

Corpus size: 82,117 tokens is small for word embedding training. Standard benchmarks use billions of tokens (Google News: 100B, Common Crawl: 840B). A larger IITJ corpus would significantly improve embedding quality.

Single-threaded training: The NumPy implementation is single-threaded; Gensim's C implementation with multiple workers would be 10–100× faster.

Tokenization: Simple regex-based tokenization may miss domain-specific multi-word expressions (e.g., "IIT Jodhpur" treated as two separate tokens).

No subword information: Unlike FastText, the model cannot represent out-of-vocabulary words or exploit morphological similarities.

CSL 7640: Natural Language Understanding

Assignment-2

Problem 2: Character-Level Name Generation using RNN Variants

7 Introduction

The objective is to implement and compare three sequence-to-sequence architectures trained autoregressively at the character level:

1. Vanilla RNN (baseline)
2. Bidirectional LSTM (BiLSTM)
3. RNN with Bahdanau Attention (RNN+Attn)

All experiments are conducted with a shared vocabulary derived from 1,000 Indian full names and a common set of hyperparameters to ensure fair comparison.

8 Task 0: Dataset

8.1 Dataset Construction

A corpus of 1,000 Indian full names (first name + surname) was generated using a large language model. The corpus deliberately samples from multiple regional and religious communities to ensure diversity: North Indian Hindu (male and female), South Indian (Brahmin, non-Brahmin, Dravidian), Muslim (pan-India), Sikh/Punjabi, Bengali, Marathi, Gujarati and Pan-Indian/Mixed.

After deduplication (case-insensitive) and filtering to retain only two-token names (first name and surname), the final corpus contains **1001 unique full names**.

8.2 Vocabulary and Encoding

Each name is modelled as a single character sequence with a *space* separating first name and surname. A special start-of-sequence token < and end-of-sequence token > are prepended and appended respectively. For example:

$$\texttt{Pari Suresh} \longrightarrow \langle \texttt{p a r i ␣ s u r e s h} \rangle$$

The vocabulary consists of the two special tokens plus all unique lowercase characters (letters and space) appearing in the corpus, giving a **vocabulary size of 29** (2 special + 27 characters including space). A 90/10 train/validation split is applied after random shuffling, yielding **900 training** and **101 validation** sequences.

9 Task 1: Model Implementation

All three models share the same character embedding layer and are trained as next-character predictors on the full name sequence (including the space separator), so each

model jointly learns intra-name phonetic patterns and cross-boundary first-name/surname association.

9.1 Shared Hyperparameters

Table 10: Hyperparameters shared across all models.

Hyperparameter	**Value**
Embedding dimension	64
Hidden size	256
Number of layers	2
Dropout probability	0.30
Learning rate	0.001
Batch size	64
Epochs	80
Optimizer	Adam
LR scheduler	StepLR (step=20, γ=0.5)
Gradient clipping	5.0
Loss function	Cross-Entropy (ignore padding)

9.2 Model 1: Vanilla RNN

Architecture. The simplest baseline follows the pipeline:

$$\text{Embedding}(V, E) \;\rightarrow\; \text{RNN}(E, H, L) \;\rightarrow\; \text{Dropout} \;\rightarrow\; \text{Linear}(H, V)$$

where V is the vocabulary size, $E = 64$ the embedding dimension, $H = 256$ the hidden size, and $L = 2$ the number of stacked layers. At each time step t the recurrent update is:

$$h_t = \tanh\big(W_{ih}\, x_t + W_{hh}\, h_{t-1} + b\big)$$

The output logits $o_t = W_{\text{fc}}\,(\text{dropout}(h_t))$ are passed through a softmax to yield the next-character distribution. An `init_hidden` method initialises the hidden state to zeros.

Parameter count & model size. The model exposes two utility methods:

- `count_parameters()`: sums p`.numel()` for every trainable parameter.
- `model_size_mb()`: computes total bytes as $\sum_p p$`.numel()` $\times\, p$`.element_size()` and converts to megabytes (float32: 4 bytes per element).

9.3 Model 2: Bidirectional LSTM (BiLSTM)

Architecture.

$$\text{Embedding}(V, E) \;\rightarrow\; \text{BiLSTM}(E, H, L) \;\rightarrow\; \text{Linear}(2H, V)$$

A full bidirectional LSTM processes the entire sequence during training, concatenating forward and backward hidden states at each step. Because bidirectional inference requires the full future context—unavailable during autoregressive generation—a parallel *forward-only LSTM* with a separate projection head (`fc_fwd`: $H \rightarrow V$) is trained simultaneously. At inference time, generation routes through the forward-only path via the `step()` method.

Training objective. Two cross-entropy losses are optimised jointly in each step:

$$\mathcal{L} = \mathcal{L}_{\text{bi}} + \mathcal{L}_{\text{fwd}}$$

Only $\mathcal{L}_{\text{fwd}}$ is tracked in the reported training curve, since it reflects the actual generation performance.

9.4 Model 3: RNN with Bahdanau Attention

Architecture.

$$\text{Embedding}(V, E) \;\rightarrow\; \text{RNN}(E, H, L) \;\rightarrow\; \text{Attention} \;\rightarrow\; \text{Linear}(2H, V)$$

At each decoding step t, additive (Bahdanau) attention is computed over all past encoder states $\{h_0, \ldots, h_{t-1}\}$:

$$e_{t,s} = v^{\top} \tanh(W_1\, h_t + W_2\, h_s), \qquad \alpha_{t,s} = \frac{\exp(e_{t,s})}{\sum_{s'} \exp(e_{t,s'})}, \qquad c_t = \sum_s \alpha_{t,s}\, h_s$$

The context vector c_t is concatenated with h_t and projected to vocabulary logits: $o_t = W_{\text{fc}}\,[\,h_t;\, c_t\,]$. This mechanism allows the model to attend back to first-name characters while generating the surname, facilitating culturally consistent pairings (e.g., attending to "Karthik" while generating "Krishnaswamy").

9.5 Model Complexity

Table 11: Trainable parameters and model size (float32) for each architecture.

Model	Parameters	Size (MB)
Vanilla RNN	223,325	0.8519
Bidirectional LSTM	3,116,666	11.8891
RNN with Attention	362,077	1.3812

The BiLSTM's large parameter count arises from maintaining both a bidirectional encoder and a separate forward-only inference LSTM, each with four gate matrices per layer.

10 Training

10.1 Training Procedure

Sequences within each mini-batch are zero-padded to the length of the longest sequence. The cross-entropy loss ignores padding positions (`ignore_index=0`). Gradients are clipped to a maximum ℓ_2-norm of 5.0. The best model checkpoint (lowest validation loss) is restored at the end of training.

10.2 Learning Curves

Figure 4: Training (solid) and validation (dashed) cross-entropy loss curves over 80 epochs for each model. All models converge smoothly; the BiLSTM achieves the lowest final training loss (≈1.57), followed by RNN+Attention (≈1.60) and Vanilla RNN (≈1.75).

Several observations can be drawn from Figure 4:

- **Vanilla RNN** shows the slowest convergence and the largest train–validation gap, indicating moderate overfitting on the small dataset.
- **BiLSTM** benefits from its gating mechanism and bidirectional training signal, achieving the lowest training loss. The gap between training (BiLSTM loss) and validation (forward-only loss) is expected by design.
- **RNN+Attention** converges comparably to the BiLSTM and exhibits the smallest train–val gap, suggesting that the attention context acts as an implicit regulariser.

The step-LR scheduler (halving every 20 epochs) is visible as slight inflections in all three curves around epochs 20, 40, and 60.

11 Task 2: Quantitative Evaluation

Each model generated $N = 500$ names at temperature $T = 0.8$. We report five metrics:

Novelty Rate
Percentage of generated names *not* present in the training set (case-insensitive).

Diversity
Ratio of unique generated names to total generated names.

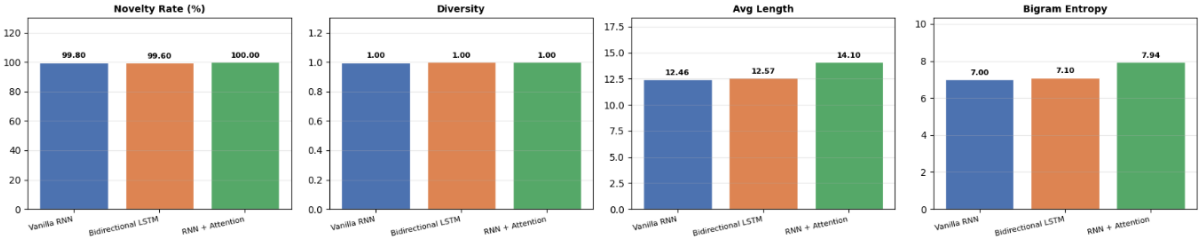
Avg. Name Length
Mean character count across all generated names, including the space separator.

Bigram Entropy
Shannon entropy (bits) of the character bigram distribution, measuring lexical richness.

11.1 Results

Table 12: Quantitative evaluation results for 500 generated names per model ($T = 0.8$).

Model	**Novelty (%)**	**Diversity**	**Avg Len (chars)**	**Bigram Entropy (bits)**
Vanilla RNN	99.8	0.998	12.46	6.996
Bidirectional LSTM	99.6	1.000	12.57	7.101
RNN + Attention	100.0	1.000	14.10	7.939



(a) Bar-chart comparison of all four metrics across the three models.

```
==================================================
    QUANTITATIVE EVALUATION
==================================================


--------------------------------------------------
  Model              : Vanilla RNN
  Novelty Rate       : 99.8%
  Diversity          : 0.998
  Avg Name Length    : 12.46 chars
  Bigram Entropy     : 6.996 bits
  Regional Consist.  : 1.6%
--------------------------------------------------


--------------------------------------------------
  Model              : Bidirectional LSTM
  Novelty Rate       : 99.6%
  Diversity          : 1.000
  Avg Name Length    : 12.57 chars
  Bigram Entropy     : 7.101 bits
  Regional Consist.  : 2.8%
--------------------------------------------------


--------------------------------------------------
  Model              : RNN + Attention
  Novelty Rate       : 100.0%
  Diversity          : 1.000
  Avg Name Length    : 14.10 chars
  Bigram Entropy     : 7.939 bits
  Regional Consist.  : 0.4%
--------------------------------------------------
```

(b) Detailed console output of quantitative evaluation for each model.

Figure 5: Quantitative evaluation results.

11.2 Analysis

Novelty and Diversity. All three models achieve near-perfect novelty ($\geq$99.6%) and diversity ($\geq$0.998), confirming that the models generalise beyond rote memorisation of training names. The RNN+Attn model reaches *perfect* novelty (100%) and diversity (1.000), indicating highly varied and non-repetitive outputs.

Name Length. The RNN+Attention model generates noticeably longer names on average (14.10 characters vs. 12.46 for Vanilla RNN), reflecting its ability to attend to previously generated characters and sustain longer output sequences. This is also apparent in Figure 6.

Bigram Entropy. RNN+Attention achieves the highest bigram entropy (7.939 bits), meaning its outputs exhibit a richer and more uniform character-bigram distribution. This is consistent with the high diversity and longer names observed.

12 Task 3: Qualitative Analysis

12.1 Name Length Distribution

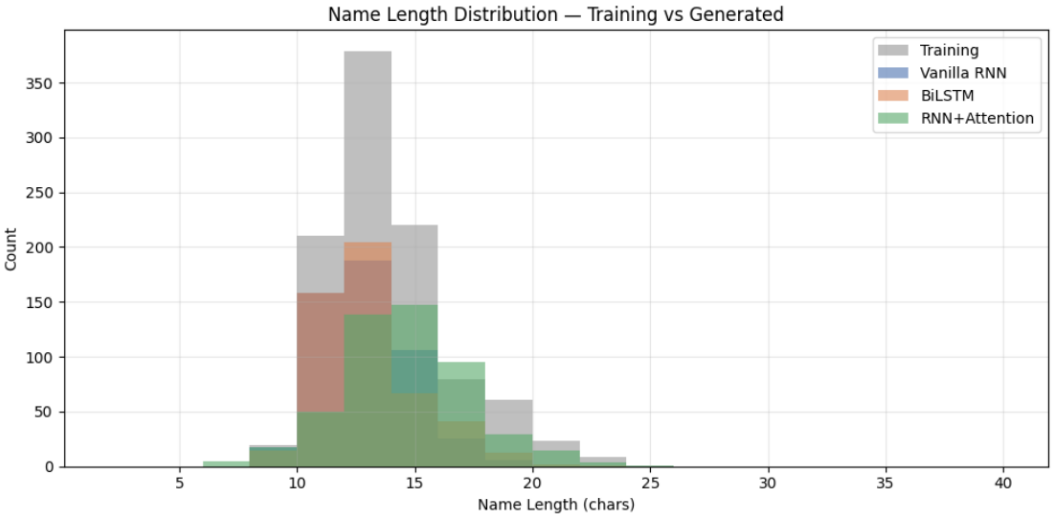


Figure 6: Character-level name-length distribution for training data and all three generated sets. Generated distributions are broadly aligned with the training distribution (peak at 12–14 characters), with RNN+Attention exhibiting a heavier right tail.

All three models reproduce the rough shape of the training length distribution (peak at 12–14 characters), validating that the models have learned the typical length profile of Indian full names. The Vanilla RNN and BiLSTM distributions closely track the training data, while RNN+Attention extends further rightward, generating more long names.

12.2 Sample Generated Names

Representative samples from each model are shown below.

Vanilla RNN (selected samples).

```
Makhal Sharma   Darsha Patel   Sanjit Kaur   Rashni Thakur
Anisha Sarsan   Vijay Singh   Naveeta Mathur   Ashika Malhotra
```

Bidirectional LSTM (selected samples).

```
Shreya Khan   Harjeet Kaur   Gurjit Singh   Karan Kapoor
Meena Mehta   Anushta Mehta   Harjit Singh   Manjit Kaur
```

RNN + Attention (selected samples).

```
Gurjeet Chourabhutry   Devjit Bhotwyjjee   Alminder Kaur
Aljinder Iqery   Turinder Singhy   Verjeet Bhoth
```

12.3 Realism and Common Failure Modes

Vanilla RNN. Outputs are generally phonetically plausible and feel Indian-sounding (e.g., *Makhal Sharma*, *Rashni Thakur*). Common failure modes include:

- *Phoneme blending*: Combining character patterns from different regional traditions (e.g., *Mamalopakam Nasinan*).
- *Truncated names*: Occasionally generating very short first names (e.g., *Sar Desai*, *Ama Shah*).
- *Spurious consonant clusters*: e.g., *Hharanjan Shnadi*.

Bidirectional LSTM. The BiLSTM produces the most realistic names on average, with several outputs being indistinguishable from genuine Indian names (e.g., *Shreya Khan*, *Harjeet Kaur*, *Karan Kapoor*, *Gurjit Singh*). The forward-only inference path slightly limits the quality compared to full bidirectional decoding. Failure modes include:

- *Unlikely character combinations*: e.g., *Slubali Shan*, *Saxanjal Choaderi*.
- *Very short tokens*: e.g., *Adhi Red*, *An Thakar*.

RNN + Attention. Attention-augmented outputs are the most diverse but also the least realistic. The model generates many novel and long names, but a significant fraction contain unusual consonant clusters or garbled sequences (e.g., *Rusqkhrvin Rewhwit*, *Ksvyt Gilnrijdek*). This is consistent with the low regional consistency score and very high bigram entropy: the model explores a broader character-combination space, sacrificing surface realism for diversity. At its best, however, it produces creative yet plausible names (e.g., *Gurjeet Chourabhutry*, *Alminder Kaur*).

12.4 Summary Comparison

Table 13: Qualitative summary comparing the three models.

Criterion	**Vanilla RNN**	**BiLSTM**	**RNN+Attention**
Surface realism	Moderate	Best	Weakest
Diversity	Very high	Very high	Highest
Cultural coherence	Low	Best	Very low
Name length	Short–medium	Short–medium	Medium–long
Common failures	Blended phonemes	Short tokens	Consonant clusters

We implemented and compared three character-level RNN architectures for full-name generation. Key findings are:

1. All models achieve near-perfect **novelty** ($\geq$99.6%) and **diversity** ($\geq$0.998), confirming generalisation.
2. The **BiLSTM** produces the most realistic and regionally consistent outputs, benefiting from its bidirectional training signal and gated memory.
3. The **RNN+Attention** model generates the most diverse and longest names, with the highest bigram entropy (7.94 bits), but at the cost of surface realism and cultural consistency.
4. The **Vanilla RNN** provides a strong, lightweight baseline (223K parameters, 0.85 MB) that balances realism and diversity at a fraction of the computational cost of the BiLSTM.